UNIBALSAS

CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

RELATÓRIO TÉCNICO

GERENCIADOR DE PERSISTÊNCIA DE DADOS EM ARQUIVOS ESTRUTURADOS (CSV E JSON)

Atividade TED 03

Disciplina: Algoritmos e Programação Avançada

Integrantes

Fernando Lemke da Silveira

Erik Alves de Sousa

BALSAS - MA

2026

SUMÁRIO

1. IDENTIFICAÇÃO 3
2. INTRODUÇÃO E OBJETIVOS 3
3. DESCRIÇÃO DA SOLUÇÃO 3
4. IMPLEMENTAÇÃO 4
5. TESTES E EVIDÊNCIAS DE EXECUÇÃO 7
6. CONCLUSÃO 13
7. REFERÊNCIAS 13

1. IDENTIFICAÇÃO

Instituição	Unibalsas
Curso	Análise e Desenvolvimento de Sistemas (ADS)
Disciplina	Algoritmos e Programação Avançada
Título da atividade	Gerenciador de Persistência de Dados em Arquivos Estruturados
Integrantes	Fernando Lemke da Silveira; Erik Alves de Sousa

2. INTRODUÇÃO E OBJETIVOS

O projeto *Gerenciador de Persistência de Dados em Arquivos Estruturados* consiste em uma aplicação de linha de comando desenvolvida em Python para manipular informações armazenadas em arquivos CSV e JSON. A solução centraliza operações de leitura, gravação, conversão de formatos, inclusão e atualização de registros, procurando preservar versões anteriores por meio de cópias de segurança antes de sobrescrever arquivos existentes.

A proposta é aplicar conceitos de manipulação de arquivos, serialização de dados, tratamento de exceções e organização de código em uma situação prática. O projeto utiliza recursos da biblioteca padrão do Python, sem dependências externas, e apresenta um menu interativo que permite executar as funcionalidades sem a necessidade de editar o código a cada operação.

Objetivo geral. Implementar um gerenciador que permita ler, converter e atualizar dados persistidos em CSV e JSON de maneira organizada e com mecanismos de proteção contra perda de dados.

Objetivos específicos. Utilizar o gerenciador de contexto `with open(...)`; ler e gravar os dois formatos; converter CSV em JSON e JSON em CSV; incluir e atualizar registros; tratar erros relacionados a arquivos, codificação e estrutura dos dados; e gerar backup automático antes de operações de escrita ou atualização.

3. DESCRIÇÃO DA SOLUÇÃO

A implementação é organizada em torno da classe `DataPersistenceManager`, responsável por concentrar as operações de persistência. Os métodos de leitura, escrita e conversão são definidos como métodos de classe ou estáticos, permitindo que o menu invoque as funcionalidades sem criar uma instância da classe.

3.1 Organização dos arquivos

A estrutura documentada no README prevê um arquivo principal `main.py`, os arquivos de dados dentro da pasta `dados/`, uma pasta `backups/` criada quando necessária, os testes automatizados em `testes/` e o arquivo de instruções `README.md`. O menu utiliza inicialmente os caminhos `dados/alunos.csv` e `dados/alunos.json`.

```
TED03_Projeto_Final_Corrigido/
├── main.py
├── dados/
│   ├── alunos.csv
│   └── alunos.json
├── backups/
├── testes/
│   └── test_main.py
└── README.md
```

Os testes ficam em testes/test_main.py. Execute a suíte na pasta principal com python -m unittest discover -s testes -v.

3.2 Principais funcionalidades

Funcionalidade	Descrição
Leitura CSV	Lê o cabeçalho e os registros com `csv.DictReader`, retornando uma lista de dicionários.
Leitura JSON	Interpreta o conteúdo com `json.load`.
CSV para JSON	Lê os registros do CSV e os serializa como uma lista de objetos JSON.
JSON para CSV	Valida a lista de objetos, reúne as chaves dos registros e grava o cabeçalho e as linhas no CSV.
Inclusão e atualização	Adiciona registros ou atualiza o primeiro registro que corresponda ao campo e valor identificadores. A classe aceita CSV e JSON; o menu oferece essas operações para CSV.
Backup e escrita segura	Copia o arquivo anterior para `backups/` e grava o novo conteúdo em arquivo temporário antes de substituir o destino.

3.3 Manipulação e persistência dos dados

No CSV, cada linha representa um registro e a primeira linha contém os nomes das colunas. A leitura por `DictReader` associa os valores às chaves correspondentes, formando dicionários. No conjunto de exemplo, os campos são *nome*, *idade* e *curso*, e os valores lidos permanecem como texto; por exemplo, a idade de João é mantida como a string `"20"`.

No JSON, os dados do exemplo são representados como uma lista de objetos, cada um contendo os mesmos campos. Ao converter JSON para CSV, o programa reúne as chaves encontradas em todos os registros, evitando descartar uma coluna que apareça apenas em um registro posterior. Os arquivos são lidos em UTF-8, com suporte a BOM por meio de `utf-8-sig`, e a gravação JSON utiliza `ensure_ascii=False` para preservar acentos.

4. IMPLEMENTAÇÃO

Esta seção apresenta trechos selecionados de `main.py`. Os fragmentos foram reduzidos ao necessário para explicar os mecanismos centrais, preservando o comportamento implementado no código enviado.

4.1 Gerenciador de contexto `with open(...)`

O comando `with open(...)` abre o arquivo dentro de um contexto controlado. Ao final do bloco, o Python fecha o arquivo automaticamente, inclusive quando ocorre uma exceção durante a operação. No método de leitura CSV, o modo é textual, a codificação é UTF-8 com tratamento de BOM e `newline=""` é utilizado em conjunto com o módulo `csv`.

```
with open(path, "r", encoding="utf-8-sig", newline="") as arquivo:
    leitor = csv.DictReader(arquivo)
    if leitor.fieldnames is None:
        raise ValueError("O CSV está vazio ou não possui cabeçalho.")
    dados = list(leitor)
```

O gerenciador de contexto também é utilizado na cópia de segurança, com modos binários (`rb` e `wb`), e na escrita de arquivos temporários, nos modos textuais de gravação. Assim, o fechamento é controlado em diferentes etapas da persistência.

4.2 Leitura, escrita e conversão de CSV

A leitura utiliza `csv.DictReader`, que transforma cada linha em um dicionário. Na gravação, `csv.DictWriter` recebe a lista de colunas, escreve o cabeçalho e grava os registros. O parâmetro `extrasaction="raise"` evita ignorar silenciosamente chaves que não estejam entre as colunas informadas.

```
with open(arquivo_temporario, "w", encoding="utf-8", newline="") as arquivo:
    escritor = csv.DictWriter(
        arquivo, fieldnames=nomes_colunas, extrasaction="raise"
    )
    escritor.writeheader()
    escritor.writerows(dados)
```

Na conversão CSV para JSON, o método `csv_para_json` chama primeiro `ler_csv`. Se a leitura falhar, a conversão é cancelada; caso contrário, a lista de dicionários é enviada para `escrever_json`, com backup habilitado.

```
dados = cls.ler_csv(caminho_csv)
if dados is None:
    print("[CONVERSÃO CANCELADA] Não foi possível ler o CSV de origem.")
    return False
return cls.escrever_json(caminho_json, dados, backup=True)
```

Na direção inversa, o programa exige que o JSON contenha uma lista de dicionários e que ela não esteja vazia. Em seguida, reúne as chaves de todos os registros para formar o cabeçalho do CSV e chama o método de escrita correspondente.

4.3 Leitura e escrita de JSON

A leitura JSON utiliza `json.load` para interpretar o conteúdo do arquivo. A escrita é realizada por `json.dump`, com indentação de quatro espaços e preservação dos caracteres acentuados. Antes de gravar, o código chama `json.dumps` para verificar se o objeto pode ser serializado.

```
with open(path, "r", encoding="utf-8-sig") as arquivo:
    dados = json.load(arquivo)

# Na gravação do arquivo temporário:
with open(arquivo_temporario, "w", encoding="utf-8") as arquivo:
    json.dump(dados, arquivo, ensure_ascii=False, indent=4)
```

4.4 Tratamento de exceções

As operações de leitura e escrita tratam erros específicos para apresentar mensagens compreensíveis e evitar o encerramento inesperado do menu. Entre as situações previstas estão arquivo inexistente, falta de permissão, codificação inválida, extensão inadequada, conteúdo JSON malformado, erro de CSV e falhas gerais de entrada e saída.

```
except FileNotFoundError:
    print(f"[ERRO] Arquivo CSV não encontrado: '{caminho}'.")
except PermissionError:
    print(f"[ERRO] Sem permissão para ler '{caminho}'.")
except UnicodeDecodeError:
    print(f"[ERRO] Codificação inválida em '{caminho}'. Salve o arquivo em UTF-8.")
except (csv.Error, ValueError) as erro:
    print(f"[ERRO CSV] Não foi possível interpretar o arquivo: {erro}")
except OSError as erro:
    print(f"[ERRO DE ARQUIVO] {erro}")
```

No caso do JSON, `json.JSONDecodeError` é tratado separadamente, permitindo informar a linha e a coluna em que a estrutura foi considerada inválida. Nas rotinas de gravação, os métodos retornam `False` quando uma falha prevista é capturada; nas rotinas de leitura, retornam `None`.

4.5 Backup automático e gravação atômica

Antes de sobrescrever um arquivo existente, `criar_backup` verifica o caminho e cria a pasta `backups` no mesmo diretório do arquivo original. O nome da cópia inclui o nome-base, um marcador de data e hora e a extensão original seguida de `.bak`. A cópia é feita em modo binário e os metadados são copiados com `shutil.copystat`. Se o arquivo de origem ainda não existir, o método informa que não há versão anterior a copiar.

```
pasta_backup = origem.parent / "backups"
pasta_backup.mkdir(parents=True, exist_ok=True)
marca_tempo = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
destino = pasta_backup / f"{origem.stem}_{marca_tempo}{origem.suffix}.bak"
with open(origem, "rb") as arquivo_origem:
    with open(destino, "wb") as arquivo_backup:
        shutil.copyfileobj(arquivo_origem, arquivo_backup)
```

Depois do backup, o método `_gravar_atomicamente` cria um arquivo temporário no mesmo diretório do destino. O gravador escreve nesse arquivo e, ao finalizar com sucesso, `os.replace` substitui o arquivo de destino. Se ocorrer uma exceção, o arquivo temporário é removido quando possível e o erro é propagado para o método chamador.

```
descritor, nome_temporario = tempfile.mkstemp(
    prefix=f".{destino.stem}_", suffix=".tmp", dir=str(destino.parent)
)
os.close(descritor)
arquivo_temporario = Path(nome_temporario)
gravador(str(arquivo_temporario))
os.replace(arquivo_temporario, destino)
```

Ordem de proteção: (1) validação dos dados e da extensão; (2) criação do backup, quando habilitado e quando já existe arquivo anterior; (3) gravação no arquivo temporário; (4) substituição do destino. Se a criação do backup falhar, a operação de escrita não prossegue, preservando a versão original.

5. TESTES E EVIDÊNCIAS DE EXECUÇÃO

Os cenários verificam leitura, conversão entre formatos, atualização de registros, backups e tratamento de erros. As operações de alteração foram realizadas em cópias de teste, preservando os dados de exemplo.

As evidências registram os resultados obtidos em cada cenário de execução do programa.

5.1 Leitura de arquivo CSV

Foi selecionada a opção 1 do menu. O método leu o arquivo `dados/alunos.csv` e apresentou três registros: João, Maria e Pedro, com os campos nome, idade e curso. O resultado confirma a leitura do cabeçalho e a associação dos valores às respectivas chaves.

Figura 1 - Leitura do arquivo CSV pelo menu da aplicação.

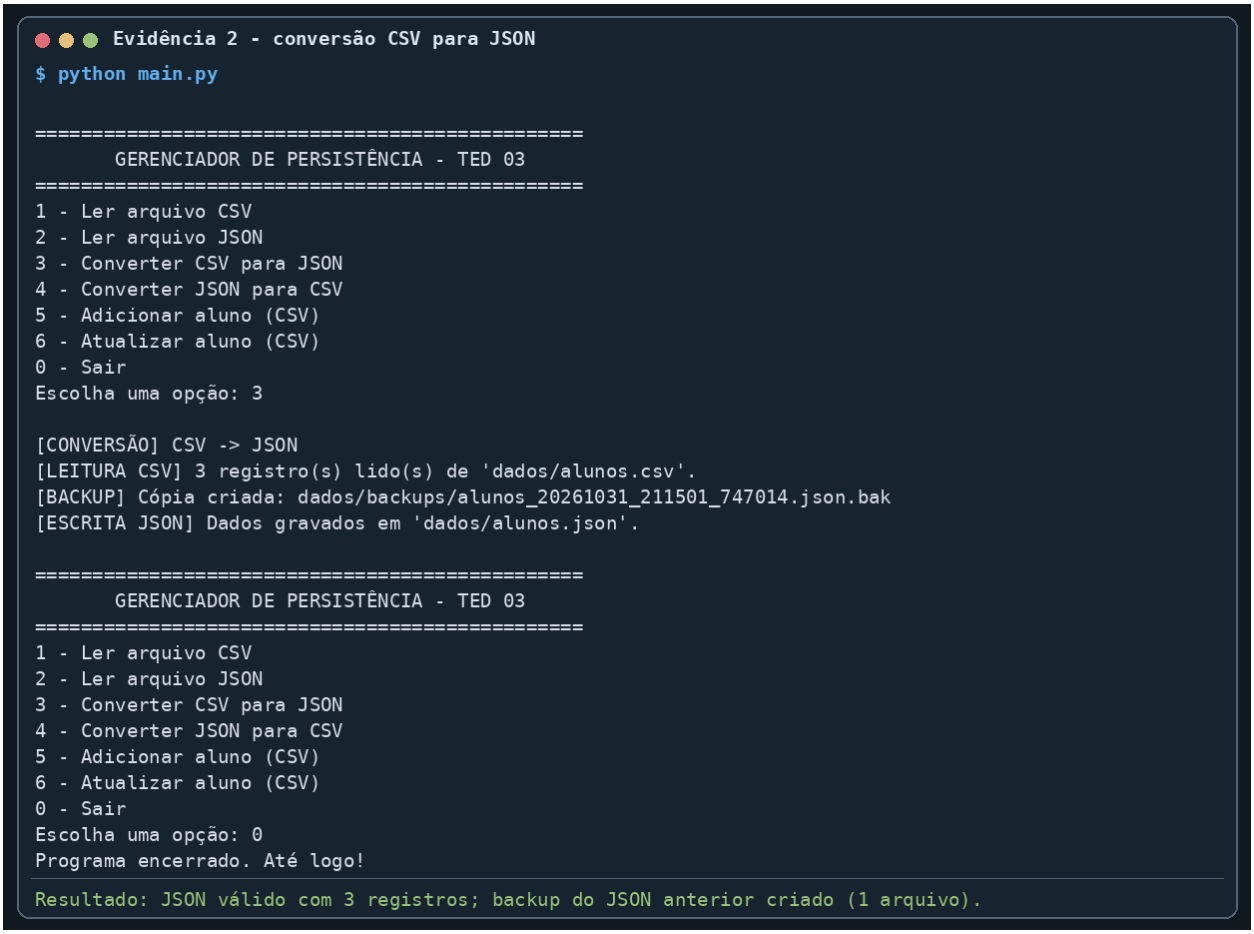


Fonte: execução de validação do projeto enviado (2026).

5.2 Conversão de CSV para JSON

Foi selecionada a opção 3. A aplicação leu os três registros do CSV, acionou a rotina de escrita JSON e gerou o arquivo de destino. Como o JSON já existia na cópia utilizada no teste, a mensagem de backup confirma que a versão anterior foi preservada antes da gravação. A validação posterior confirmou uma lista JSON com três objetos.

Figura 2 - Conversão CSV para JSON, com backup do arquivo JSON preexistente.

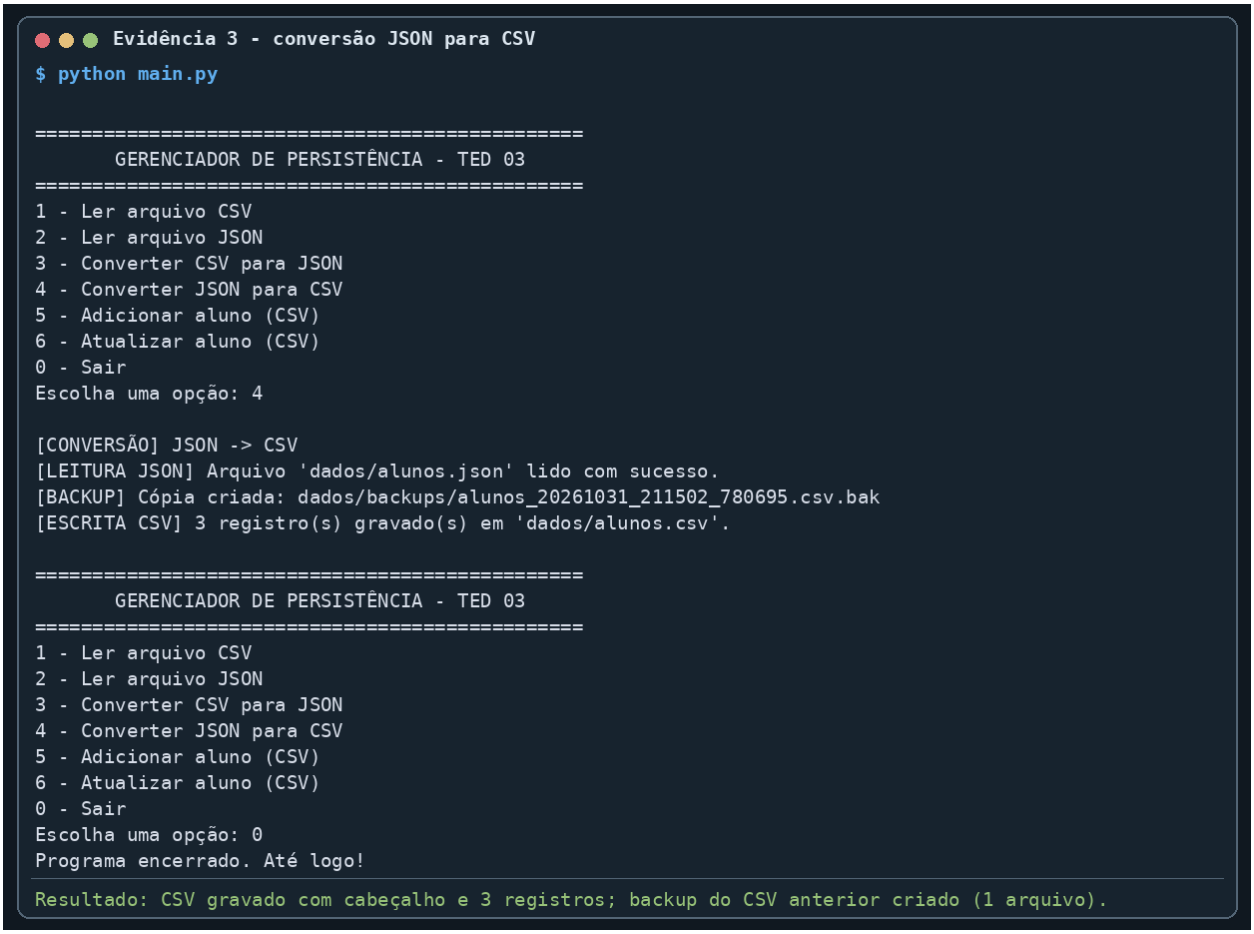


Fonte: execução de validação do projeto enviado (2026).

5.3 Conversão de JSON para CSV

Foi selecionada a opção 4. O programa leu a lista de objetos JSON, reuniu os nomes dos campos e gravou o CSV correspondente. O arquivo CSV já existia na cópia de teste; por isso, a saída também registra a criação de uma cópia de segurança antes da escrita. A validação do arquivo final confirmou o cabeçalho e os três registros.

Figura 3 - Conversão JSON para CSV e confirmação de backup do CSV anterior.

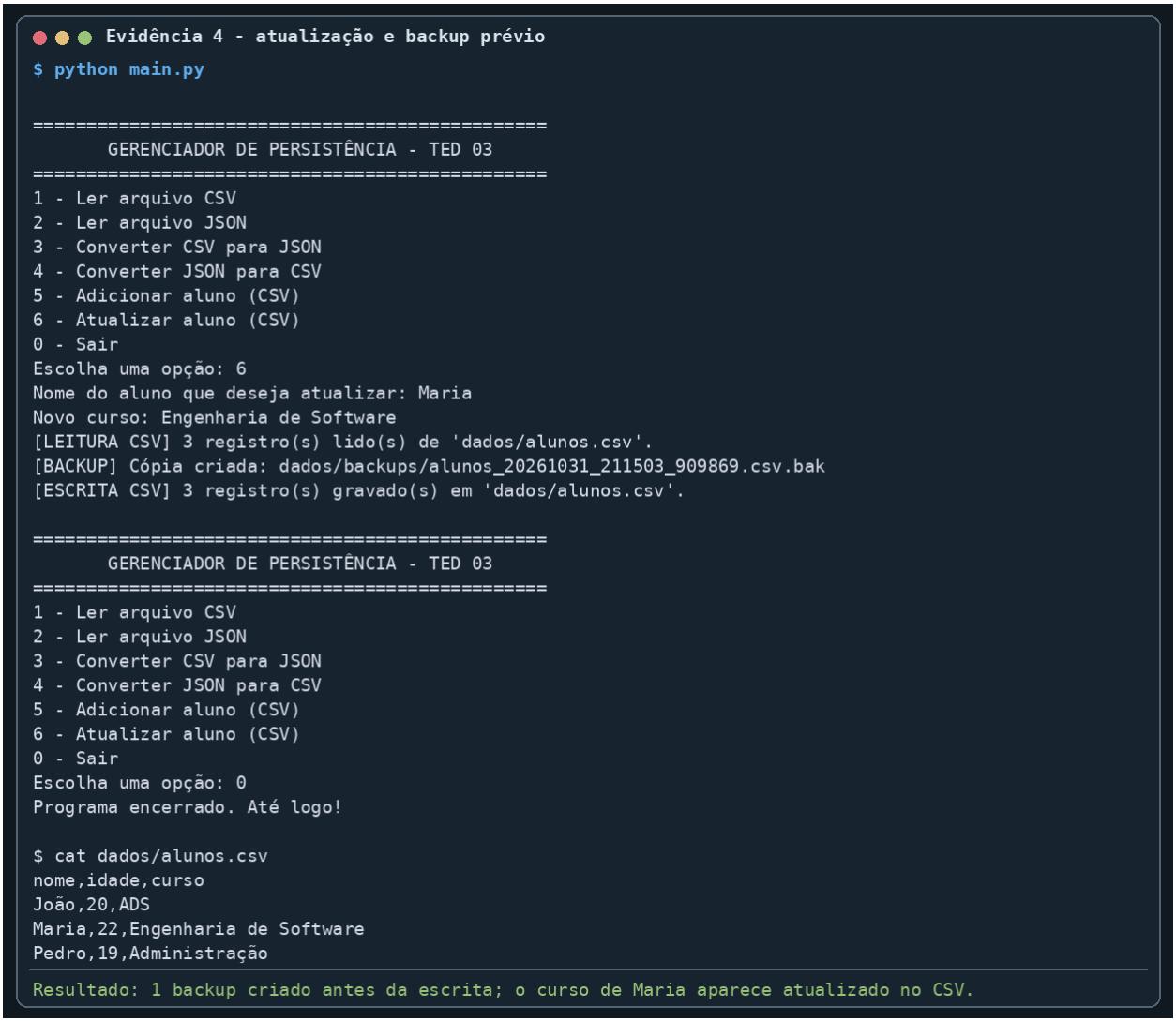


Fonte: execução de validação do projeto enviado (2026).

5.4 Atualização de registro com backup prévio

Foi utilizada a opção 6 para localizar o registro cujo campo *nome* corresponde a “Maria” e alterar o campo *curso* para “Engenharia de Software”. A saída mostra a leitura dos três registros, a criação do backup e a gravação do CSV atualizado. A consulta posterior ao arquivo confirma que o curso de Maria foi alterado e que João e Pedro permaneceram com os valores anteriores.

Figura 4 - Atualização do curso de Maria, backup automático e conferência do CSV resultante.

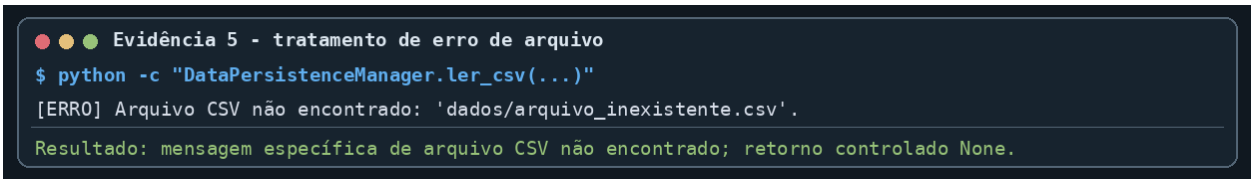


Fonte: execução de validação do projeto enviado (2026).

5.5 Tratamento de erro de arquivo

Para demonstrar a situação de erro, foi solicitada a leitura de `dados/arquivo_inexistente.csv`. A aplicação capturou `FileNotFoundError`, exibiu uma mensagem específica e retornou `None`, sem apresentar um rastreamento de exceção não tratada.

Figura 5 - Mensagem apresentada para arquivo CSV inexistente.



Fonte: execução de validação do projeto enviado (2026).

5.6 Testes automatizados

O arquivo testes/test_main.py contém sete testes automatizados: leitura CSV, conversão CSV para JSON, conversão JSON para CSV, criação de backup antes de sobrescrever, inclusão de registro, atualização de registro e leitura de arquivo inexistente. A execução foi realizada a partir da pasta principal do projeto pelo comando python -m unittest discover -s testes -v. Os sete testes foram concluídos com sucesso, conforme a Figura 6.

Figura 6 - Resultado dos sete testes automatizados.



Fonte: execução dos testes automatizados do projeto (2026).

6. CONCLUSÃO

A execução e a análise do código demonstraram que os objetivos centrais do projeto foram alcançados. A aplicação realiza a leitura e a escrita de arquivos CSV e JSON, converte dados entre os dois formatos, permite inclusão e atualização de registros e apresenta um menu de linha de comando para acionar as operações. O uso de `with open(...)` organiza o ciclo de vida dos arquivos, enquanto o tratamento de exceções apresenta mensagens específicas para diferentes falhas.

O mecanismo de backup automático preserva uma cópia do arquivo existente antes da sobrescrita. Na sequência, a gravação ocorre em um arquivo temporário e a substituição do destino é feita por `os.replace`, reduzindo o risco de deixar o arquivo principal parcialmente gravado. Nos testes realizados, foi possível confirmar a criação do backup antes da atualização e das conversões que sobrescreveram arquivos existentes.

Os sete testes automatizados foram executados e aprovados. A suíte contempla leitura CSV, conversão CSV para JSON e JSON para CSV, criação de backup antes de sobrescrever, inclusão e atualização de registros e leitura de arquivo inexistente. Como melhoria futura, podem ser incluídos testes específicos para falhas de permissão e codificação.

Durante a verificação, observou-se que a aplicação preserva os valores do CSV como texto, sem conversão automática de tipos numéricos. Esse comportamento é compatível com a leitura por csv.DictReader e deve ser considerado caso futuras versões precisem realizar cálculos com campos como idade. No conjunto analisado, os exemplos de dados foram lidos e convertidos sem perda dos nomes, cursos ou acentos.

Conclui-se que a solução atende ao escopo acadêmico proposto e demonstra, de forma prática, conceitos importantes de persistência de dados, conversão de formatos, manipulação segura de arquivos e tratamento de exceções em Python.

7. REFERÊNCIAS

LEMKE DA SILVEIRA, Fernando; SOUSA, Erik Alves de. Gerenciador de Persistência de Dados em Arquivos Estruturados (CSV e JSON): TED 03. Código-fonte Python (main.py), arquivos de exemplo (alunos.csv e alunos.json) e testes automatizados (test_main.py). Material de projeto acadêmico, 2026.

LEMKE DA SILVEIRA, Fernando; SOUSA, Erik Alves de. TED 03 - Gerenciador de Persistência de Dados (CSV e JSON). README.md do projeto. Material técnico de apoio à execução e organização do projeto, 2026.

PYTHON SOFTWARE FOUNDATION. unittest — Unit testing framework. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/unittest.html. Acesso em: 31 out. 2026.